Содержание

1 Задача 3.1.6d **1**

2 Псевдокод **1**

3 Решение **1**

4 Сложность **5**

1 Задача 3.1.6d

Условие: Отсортировать последовательность $a_1, \ldots, a_n$ в порядке возрастания элементов. Выбрана сортировка HeapSort.
in: $n, a_1, \ldots, a_n$
out: $b_1, \ldots, b_n$, где $b_1 \leq b_2 \leq \ldots \leq b_n$

2 Псевдокод

Псевдокод HeapSort, приведённый в исходном файле:

```
public static void sort(Comparable[] pq) {
    int n = pq.length;
    for (int k = n/2; k >= 1; k--)
        sink(pq, k, n);
    int k = n;
    while (k > 1) {
        exch(pq, 1, k--);
        sink(pq, 1, k);
    }
}

private static void sink(Comparable[] pq, int l, int m) {
    while (2*l <= m) {
        int j = 2*l;
        if (j < m && less(pq, j, j+1)) j++;
        if (!less(pq, l, j)) break;
        exch(pq, l, j);
        l = j;
    }
}
```

3 Решение

```
; Data Model
;0: r0
;1: trash | iterator
;2: &end
;3: n
;4: k
```

```
;5: l
;6: m
;7: j
;8: call (from what place was sink called last time)
;9 ... 9+n-1: arr

Load =0
Read 3
Sub 3           ; r3 := n
JZero END
JGTZ END  ; n <= 0 ==> exit (invalid size)

Load 3
Add =9
Store 2   ; r2 := 9 + n (arr_a.end())

Load =9
Store 1   ; r1 := 9 (arr_a.begin())

READ_ARRAY: Load 2
 Sub 1
 JZero SORT  ; if r2 == r1 exit read
 Read *1
 Load 1
 Add =1
 Store 1  ; r1++
 Jump READ_ARRAY
SORT: Load 3
    Div =2
    Store 4     ; r4 := k = n // 2
FOR: Load =1
    Sub 4
    JGTZ FOR_END    ; if k < 1 then exit for
    Load 4
    Store 5
    Load 3
    Store 6     ; l := k, m := n
    Load =0
    Store 8     ; call = 0 (call from for)
    Jump SINK   ; call sink(k, n)
FOR_CALL_END: Load 4
    Sub =1
    Store 4     ; k--
    Jump FOR
FOR_END: Load 3
    Store 4     ; k := n
WHILE_k_gt_1: Load =1
    Sub 4
    JZero PRINT_INIT
    JGTZ PRINT_INIT   ; if k <= 1 then exit while and exit sort
```

```
    ; SWAP
    Load 4
    Add =8
    Load *0
    Store 1      ; r1 := arr[k]
    Load 9       ; r0 := arr[1]
    Store 2      ; r2 := arr[1]
    Load 1
    Store 9      ; arr[1] := arr[k]
    Load 4
    Add =8
    Store 1      ; r1 := &arr[k]
    Load 2
    Store *1     ; arr[k] (= *r1) := arr[1] (= r2)
    ; k--
    Load 4
    Sub =1
    Store 4
    ; CALL sink(1, k)
    Load =1
    Store 5      ; l := 1
    Load 4
    Store 6      ; m := k
    Load =1
    Store 8      ; call := 1 (call sink from while)
    Jump SINK
SINK: Load =2
    Mult 5
    Sub 6
    JGTZ CALL_RETURN    ; if 2*l-m > 0 then exit SINK
    Load =2
    Mult 5
    Store 7      ; j := 2*l
    ; if (j < m && less(pq, j, j+1)) j++;
    Load 7
    Sub 6
    JZero IF_BREAK
    JGTZ IF_BREAK       ; if j-m>=0 then do not do IF
    Load 7
    Add =8
    Store 1      ; r1 := &arr[j]
    Add =1
    Store 2      ; r2 := &arr[j+1]
    Load *1
    Sub *2
    JZero IF_BREAK
    JGTZ IF_BREAK       ; if a[j] >= a[j+1] then do not do IF
    Load 7
    Add =1
    Store 7      ; j++
```

```
; if (!less(pq, l, j)) break;
IF_BREAK: Load 7
    Add =8
    Load *0
    Store 1     ; r1 := arr[j]
    Load 5
    Add =8
    Load *0     ; r0 := arr[l]
    Sub 1       ; r0 := arr[l] - arr[j]
    JZero CALL_RETURN
    JGTZ CALL_RETURN    ; if a[l] >= a[j] then return from SINK
    ; SWAP if not break executed
    Load 7
    Add =8
    Load *0
    Store 1     ; r1 := arr[j]
    Load 7
    Add =8      ; r0 := &arr[j]
    Store 2     ; r2 := &arr[j]
    Load 5
    Add =8
    Load *0     ; r0 := arr[l]
    Store *2    ; arr[j] (= *r2) := arr[l] (= r0)
    Load 5
    Add =8
    Store 2     ; r2 := &arr[l]
    Load 1      ; r0 := arr[j] (previous)
    Store *2    ; arr[l] (= *r2) := arr_previous[j] (= r0)
    ; l = j
    Load 7
    Store 5
    ; goto WHILE
    Jump SINK
CALL_RETURN: Load 8
    JZero FOR_CALL_END  ; return from sink(k, n)
    Jump WHILE_k_gt_1   ; return from sink(1, k)
PRINT_INIT: Load =9
    Store 1     ; r1 := arr.begin()
    Add 3
    Store 2     ; r2 := arr.end()
PRINT: Load 2
    Sub 1
    JZero END   ; if r1 == r2 then exit print
    Write *1
    Load 1
    Add =1
    Store 1     ; r1++
    Jump PRINT
END: Halt 0
```

4 Сложность

Допустимый размер входа: $n \geq 1$. Каждая команда RAM считается за один шаг. Пусть $h = \lfloor n/2 \rfloor$.

Часть программы вне вызовов `SINK` занимает точно:

1. Проверка входа — 5 шагов.
2. Инициализация указателей массива — 5 шагов.
3. Чтение массива — $8n + 3$ шага.
4. Инициализация построения кучи — 3 шага.
5. Управление циклом построения кучи, без вызовов `SINK` — $14h + 5$ шагов.
6. Обмен элементов и управление циклом извлечения максимума, без вызовов `SINK` — $27n - 24$ шага.
7. Инициализация вывода, вывод массива и завершение программы — $8n + 8$ шагов.

Итого фиксированная часть:

$$T_{\text{fixed}}(n) = 43n + 14 \left\lfloor \frac{n}{2} \right\rfloor + 5.$$

Кроме того, выполняются h вызовов `SINK` при построении кучи и $n - 1$ вызовов при извлечении максимума. Обозначим через $C_r(A, l, m)$ точное количество команд одного вызова `SINK` для текущего состояния массива A, где $r = 0$ означает возврат в цикл построения кучи, а $r = 1$ — возврат в цикл извлечения максимума.

Точное число шагов одного вызова можно задать рекуррентно. Если $2l > m$, то

$$C_r(A, l, m) = 6 + r.$$

Иначе положим $j = 2l$. Выбранный потомок q и затраты B на проверку наличия правого потомка и выбор большего потомка определяются так:

$$(q, B) = \begin{cases} (j, 3), & j = m, \\ (j, 4 + 8), & j < m,\ A_j = A_{j+1}, \\ (j, 4 + 9), & j < m,\ A_j > A_{j+1}, \\ (j + 1, 4 + 12), & j < m,\ A_j < A_{j+1}. \end{cases}$$

Здесь затраты 8, 9 и 12 учитывают разные исходы сравнения двух потомков; случай 12 включает увеличение индекса потомка. Пусть A' — массив после обмена элементов в позициях l и q. Тогда

$$C_r(A, l, m) = \begin{cases} 7 + B + p + 2 + r, & A_l \geq A_q, \\ 7 + B + 10 + 19 + C_r(A', q, m), & A_l < A_q, \end{cases}$$

где $p = 9$, если $A_l = A_q$, и $p = 10$ в остальных случаях. Эта запись считает шаги по выполненным ветвям, а не перечисляет команды по одной.

Точное число шагов для конкретного входа имеет вид

$$T(n, A) = 43n + 14 \left\lfloor \frac{n}{2} \right\rfloor + 5 + \sum_{k=1}^{h} C_0(A^{(k)}, k, n) + \sum_{m=1}^{n-1} C_1(A^{[m]}, 1, m),$$

где $A^{(k)}$ и $A^{[m]}$ — состояния массива непосредственно перед соответствующими вызовами `SINK`. Следовательно, точное число шагов в худшем случае задаётся формулой

$$T_{\max}(n) = \max_{A} T(n, A),$$

где максимум берётся по допустимым входным последовательностям длины n.

Количество шагов внутри `SINK` зависит от данных: от того, сколько уровней проходит элемент, имеется ли правый потомок и какой из двух потомков выбирается. Поэтому для вычисления максимума требуется отдельно максимизировать эти ветвления с учётом изменений массива между вызовами. Ниже приведена явная верхняя оценка, достаточная для определения порядка роста.

Для оценки худшего случая заметим следующее:

- При построении кучи число переходов вниз не превышает

$$H(n) = \sum_{k=1}^{\lfloor n/2 \rfloor} \left\lfloor \log_2 \frac{n}{k} \right\rfloor = n - s_2(n),$$

  где $s_2(n)$ — число единиц в двоичной записи n.

- При извлечении максимума число переходов вниз не превышает

$$D(n) = \sum_{m=1}^{n-1} \lfloor \log_2 m \rfloor.$$

- Один переход вниз вместе с соответствующими проверками занимает не более 52 шагов, а завершающая часть одного вызова `SINK` — не более 36 шагов.

Следовательно, получаем явную верхнюю оценку:

$$\begin{aligned} T_{\max}(n) \le{}& 43n + 14 \left\lfloor \frac{n}{2} \right\rfloor + 5 \\ &+ 52\big(H(n) + D(n)\big) + 36 \left(\left\lfloor \frac{n}{2} \right\rfloor + n - 1 \right). \end{aligned}$$

Так как $H(n) = O(n)$ и $D(n) = O(n \log n)$, худшее время работы HeapSort составляет $O(n \log n)$. Для сортировки сравнением в худшем случае необходимо $\Omega(n \log n)$ сравнений, поэтому

$$\boxed{T_{\max}(n) = \Theta(n \log n)}.$$

При стандартном предположении о равномерно случайной перестановке различных элементов среднее время HeapSort также составляет $\Theta(n \log n)$.